

Project Documentation: AI-Based Siren Direction Detection

Group name: Maeron

1. Project Overview

For this challenge, our goal was to build an embedded AI system capable of recognizing the spatial direction of an emergency vehicle coming to an intersection based on its siren sound. To get the best results and build a robust system, we focused on training a neural network and correcting AI uncertainties using empirical post-processing. Our final solution was successfully trained, converted, and deployed onto the Infineon PSOC Edge AI Kit using DEEPCRAFT Studio and Eclipse ModusToolbox.

2. Data Acquisition & AI Model Training

We utilized DEEPCRAFT Studio for our core machine learning pipeline.

Data Collection: We flashed the required data collection firmware onto the PSoC Edge board and recorded a standard ambulance siren from 5 distinct angles with 10 recordings from each angle using the Live Data Collection example in DEEPCRAFT Studio.

Spatial mapping constraint (180 degrees): We deliberately restricted our spatial mapping and data collection to a 180-degree subfield due to the physical limitations of the hardware. The PSoC Edge board uses a linear array of only two microphones. While an AI could theoretically attempt to differentiate angles based on tiny amplitude variations and frequency changes caused by physical features of the board (like pin headers and USB inputs), these features are highly susceptible to distortion. Because of this, we decided to train the AI to differentiate angles based on the Time Difference of Arrival (TDOA) and Amplitude Difference between the two microphones. However, because a sound wave arriving from 45 degrees in front produces the exact same TDOA as a mirrored sound behind it, the neural network cannot reliably distinguish between the front and rear hemispheres. By constraining our model's scope to a 180-degree half-circle, we eliminated these errors.

Preprocessing: We utilized a sliding window approach to chunk the audio into processable frames. We aligned the window shape and stride in DEEPCRAFT to ensure a data pipeline from the microphones to the neural network.

Solving Unexpected Problems: During this phase, we ran into a few tricky issues that took up a lot of time. For example, hidden whitespaces in our data files caused the AI training to fail without any debugging information, and we had to spend time trying to fix it. Even stranger, when we added more audio recordings, the AI models started performing worse. To solve this, we utilized DEEPCRAFT's automated AutoML strategy, which automatically generates four distinct model variations with different network depths and optimization trade-offs. We systematically experimented and evaluated the different generated models until we found the specific model configuration that offered the best accuracy on our real-life tests. This process showed us how important it is to test everything carefully rather than just assuming more data equals better results.

3. Signal Processing & Experimental Proximity Tracking

Experimental Smoothing & Proximity Detection: During our development phase, we prototyped a measurement of the moving average of the acoustic energy of the sound reaching the PSoC board to smooth out the natural peaks and valleys of the siren's volume cycle. While we ultimately decided not to include this feature in the final, deployed code, this testing phase revealed a very useful concept. We observed that by calculating a rolling average of the volume, the system gained the potential to detect proximity trends. By comparing consecutive smoothed frames, we proved the concept that the acoustic energy could reliably indicate whether the emergency vehicle was physically approaching or moving away from the sensor, which would be helpful in the real-life scenario presented on the first day of the Hackathon.

Challenges faced: During this phase, we ran into a major challenge when trying to implement distance tracking. Our initial goal was to measure the physical distance between the phone's speaker and the evaluation board. However, the ambulance audio track we used was a recording of a vehicle passing by, meaning the siren's volume grew louder, and faded out on its own. While this variation allowed us to detect the relative direction. Meaning that we could tell exactly when the ambulance was getting closer during the soundtrack and when it was moving away. But it made calculating the absolute distance of the phone's speaker incredibly difficult. To fix this properly, we would have needed to gather a completely new dataset and retrain our AI models from scratch. Because our schedule was already very tight, a full retraining phase would have put us under too much time pressure, so we had to work within these data limitations.

4. Innovation: Rule-Based Post-Processing & Thresholding

Our main addition to the project was applied after the AI training phase, directly on the hardware. During empirical testing of the deployed model, we observed that the raw neural network output exhibited varying confidence levels depending on the specific angle. For example, the directions northeast and northwest were getting priority over east and west respectively. Instead of treating the AI as a black box, we implemented a Rule Based Post-Processing Layer in our “pdm_data_process” logic.

Empirical Thresholding: We abandoned the use of a single, static confidence threshold of 65 percent. Through systematic testing, we mapped the model's performance footprint in the real world.

Dynamic Adjustments: For directions that the AI-model historically predicted with lower raw confidence (east and west), we explicitly lowered the acceptance threshold in our C code.

Why we did this: We implemented this adjustment because we thought that the model trained in an ideal software environment faced noise and signal variations when deployed on physical hardware in the real world. By combining the neural network's predictions with conditional C-programming logic, we created a more stable system that accounts for these environmental differences.

5. Conclusion

By combining a sliding-window neural network and an empirically tuned post-processing step, we built a highly accurate application. The system successfully bypasses the physical limitations of a two-microphone array and operates effectively on the PSoC Edge evaluation kit. This project also taught us a great lesson about planning. We realized that when working with AI, unexpected bugs and delays can pop up out of nowhere at any time. Thankfully we were able to solve all our technical issues and finish the project right on time. Our experimental phase also successfully mapped out a clear architectural path for adding proximity and distance tracking in future iterations.